

AI ASSISTED CODING

ASSIGNMENT- 12.1

Name: G. Likhitha Rao

HT No:2303A52487

BT No:35

Task Description #1 (Sorting – Merge Sort Implementation)

- Task: Use AI to generate a Python program that implements the Merge Sort algorithm.
- Instructions:
 - o Prompt AI to create a function `merge_sort(arr)` that sorts a list in ascending order.
 - o Ask AI to include time complexity and space complexity in the function docstring.
 - o Verify the generated code with test cases.
- Expected Output:
 - o A functional Python script implementing Merge Sort with proper documentation.

```
... Welcome mergesort.py
C: > Users > varshitha > OneDrive > Desktop > AI Assistant coding > mergesort.py > merge
1 # Merge Sort Algorithm Implementation in Python and given time and space complexity
2 def merge_sort(arr):
3     """
4     Sorts an array using the merge sort algorithm.
5
6     Time Complexity: O(n log n) - all cases (best, average, worst)
7     Space Complexity: O(n) - for the temporary arrays used during merging
8
9     Args:
10    |   arr: List of comparable elements
11
12    Returns:
13    |   Sorted list
14    """
15    if len(arr) <= 1:
16        return arr
17
18    mid = len(arr) // 2
19    left = merge_sort(arr[:mid])
20    right = merge_sort(arr[mid:])
21
22    return merge(left, right)
23
24
25 def merge(left, right):
26     """
27     Merges two sorted arrays into a single sorted array.
28
29     Time Complexity: O(n + m) where n and m are lengths of left and right
30     Space Complexity: O(n + m) for the result array
31     """
32     result = []
33     i = j = 0
34
35     while i < len(left) and j < len(right):
36         if left[i] <= right[j]:
37             result.append(left[i])
```

```

25 def merge(left, right):
26     while i < len(left) and j < len(right):
27         if left[i] <= right[j]:
28             result.append(left[i])
29             i += 1
30         else:
31             result.append(right[j])
32             j += 1
33
34     result.extend(left[i:])
35     result.extend(right[j:])
36     return result
37
38 # Example usage:
39 if __name__ == "__main__":
40     arr = [38, 27, 43, 3, 9, 82, 10]
41     sorted_arr = merge_sort(arr)
42     print("Sorted array:", sorted_arr)
43
44
45

```

OUTPUT:

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\varshitha> & C:/Users/varshitha/anaconda3/python.exe "c:/Users/varshitha/OneDrive/Desktop/AI Assistant coding/mergesort.py"
Sorted array: [3, 9, 10, 27, 38, 43, 82]
PS C:\Users\varshitha>

```

Task Description #2 (Searching – Binary Search with AI Optimization)

- Task: Use AI to create a binary search function that finds a target element in a sorted list.
- Instructions:
 - o Prompt AI to create a function `binary_search(arr, target)` returning the index of the target or -1 if not found.
 - o Include docstrings explaining best, average, and worst-case complexities.
 - o Test with various inputs.
- Expected Output:
 - o Python code implementing binary search with AI- generated comments and docstrings.

```
Welcome  binarysearch.py X
C: > Users > varshitha > OneDrive > Desktop > AI Assistant coding > binarysearch.py > ...
1  #generate a code for binary search in python and Include docstrings explaining best, average, and
2  #most-case complexities. Test with various inputs.
3  def binary_search(arr, target):
4      """
5      Perform binary search on a sorted array.
6
7      Parameters:
8      arr (list): A sorted list of elements to search through.
9      target: The element to search for.
10
11     Returns:
12     int: The index of the target element if found, otherwise -1.
13
14     Complexity:
15     Best Case: O(1) - when the target is at the middle of the array.
16     Average Case: O(log n) - when the target is not at the middle but still found in the array.
17     Worst Case: O(log n) - when the target is not found in the array or is at one of the ends.
18     """
19     left, right = 0, len(arr) - 1
20
21     while left <= right:
22         mid = left + (right - left) // 2
23
24         # Check if the target is present at mid
25         if arr[mid] == target:
26             return mid
27         # If target is greater, ignore the left half
28         elif arr[mid] < target:
29             left = mid + 1
30         # If target is smaller, ignore the right half
31         else:
32             right = mid - 1
33
34     # Target was not found in the array
35     return -1
36
37     # Test cases
38 if __name__ == "__main__":
```

```
39     # Test cases
40     if __name__ == "__main__":
41         # Test with various inputs
42         arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
43         target = 5
44         print(f"Searching for {target} in {arr}: Index = {binary_search(arr, target)}")
45
46         target = 1
47         print(f"Searching for {target} in {arr}: Index = {binary_search(arr, target)}")
48
49         target = 10
50         print(f"Searching for {target} in {arr}: Index = {binary_search(arr, target)}")
51
52         target = 11
53         print(f"Searching for {target} in {arr}: Index = {binary_search(arr, target)}")
54
55         target = 0
56         print(f"Searching for {target} in {arr}: Index = {binary_search(arr, target)}")
57
58         target = 6
59         print(f"Searching for {target} in {arr}: Index = {binary_search(arr, target)}")
```

OUTPUT:

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\varshitha>
PS C:\Users\varshitha> & C:\Users\varshitha\anaconda3\python.exe "c:/Users/varshitha/OneDrive/Desktop/AI Assistant coding/binarysearch.py"
Searching for 5 in [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]: Index = 4
Searching for 10 in [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]: Index = 9
Searching for 11 in [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]: Index = -1
Searching for 0 in [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]: Index = -1
Searching for 6 in [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]: Index = 5

```

Task Description #3 (Real-Time Application – Inventory Management System)

- Scenario: A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:
 1. Quickly search for a product by ID or name.
 2. Sort products by price or quantity for stock analysis.
- Task:
 - o Use AI to suggest the most efficient search and sort algorithms for this use case.
 - o Implement the recommended algorithms in Python.
 - o Justify the choice based on dataset size, update frequency, and performance requirements.
- Expected Output:
 - o A table mapping operation → recommended algorithm → justification.
 - o Working Python functions for searching and sorting the inventory.

AI.py AI 7.py AI 9.py lab exam.py lab exam2.py que2.py Ass-12.5.py

```

267  ## implement a retail store's inventory system contains thousands of products, each with attributes like product ID
268  # name, price, and stock quantity. Store staff need to: Quickly search for a product by ID or name. Sort products
269  # price or quantity for stock analysis. Implement the recommended algorithms in Python. Justify the choice based
270  # on dataset size, update frequency, and performance requirements.
271  class Product:
272      def __init__(self, product_id, name, price, stock_quantity):
273          self.product_id = product_id
274          self.name = name
275          self.price = price
276          self.stock_quantity = stock_quantity
277  class InventorySystem:
278      def __init__(self):
279          self.products = []
280
281      def add_product(self, product_id, name, price, stock_quantity):
282          product = Product(product_id, name, price, stock_quantity)
283          self.products.append(product)
284
285      def search_product_by_id(self, product_id):
286          for product in self.products:
287              if product.product_id == product_id:
288                  return product
289          return None
290
291      def search_product_by_name(self, name):
292          for product in self.products:
293              if product.name.lower() == name.lower():
294                  return product
295          return None
  
```

```

Ass-12.5.py > ...
77 class InventorySystem:
96
97     def sort_products_by_price(self):
98         self.products.sort(key=lambda x: x.price)
99
100    def sort_products_by_stock_quantity(self):
101        self.products.sort(key=lambda x: x.stock_quantity)
102
103    # Example usage
104    inventory = InventorySystem()
105    inventory.add_product("P001", "Laptop", 999.99, 10)
106    inventory.add_product("P002", "Smartphone", 499.99, 20)
107    inventory.add_product("P003", "Headphones", 199.99, 15)
108    print("Search for product ID P001:", inventory.search_product_by_id("P001"))
109    print("Search for product name 'Smartphone':", inventory.search_product_by_name("Smartphone"))
110    inventory.sort_products_by_price()
111    print("Products sorted by price:", inventory.products)
112    inventory.sort_products_by_stock_quantity()
113    print("Products sorted by stock quantity:", inventory.products)
114
115    #justification for selected algorithm:
116    # For searching products by ID or name, we can use a linear search since the number of products may not
117    # be very large, and it allows us to find the product without needing to sort the data first. The time
118    # complexity of linear search is O(n) in the worst case. For sorting products by price or stock quantity,
119    # we can use the built-in sort method which implements Timsort. Timsort is efficient for real-world data and
120    # has a time complexity of O(n log n) in the worst case and O(n) in the best case when the data is already sorted
121    # . This makes it suitable for our retail store's inventory system where we may often have partially sorted data
122    # and need efficient sorting for stock analysis.

```

OUTPUT:

```

PS C:\Users\reddy\OneDrive\Desktop\AI> & C:\Users\reddy\AppData\Local\Microsoft\WindowsApps\python3.13.exe c:
/Users/reddy/OneDrive/Desktop/AI/Ass-12.5.py
Search for product ID P001: <__main__.Product object at 0x000001FE01CA6BA0>
Search for product name 'Smartphone': <__main__.Product object at 0x000001FE01F48A50>
Products sorted by price: [<__main__.Product object at 0x000001FE01F48B90>, <__main__.Product object at 0x00
001FE01F48A50>, <__main__.Product object at 0x000001FE01CA6BA0>]
Products sorted by stock quantity: [<__main__.Product object at 0x000001FE01CA6BA0>, <__main__.Product object
at 0x000001FE01F48B90>, <__main__.Product object at 0x000001FE01F48A50>]
PS C:\Users\reddy\OneDrive\Desktop\AI>

```

Task description #4: Smart Hospital Patient Management System

A hospital maintains records of thousands of patients with details such as patient ID, name, severity level, admission date, and bill amount. Doctors and staff need to:

1. Quickly search patient records using patient ID or name.
 2. Sort patients based on severity level or bill amount for prioritization and billing. Student Task
- Use AI to recommend suitable searching and sorting algorithms.
 - Justify the selected algorithms in terms of efficiency and suitability.
 - Implement the recommended algorithms in Python.

Ass-12.5.py > Patient

```
59  ##implement A hospital maintains records of thousands of patients with detailssuch as patient ID, name,
60  # severity level, admission date, and bill amount. Doctors and staff need to: Quickly search patient records
61  # using patient ID or name. Sort patients based on severity level or bill amount for prioritization and billing.
62  #select the optimal algorithm and justify i
63  class Patient:
64      def __init__(self, patient_id, name, severity_level, admission_date, bill_amount):
65          self.patient_id = patient_id
66          self.name = name
67          self.severity_level = severity_level
68          self.admission_date = admission_date
69          self.bill_amount = bill_amount
70  class HospitalRecords:
71      def __init__(self):
72          self.patients = []
73
74      def add_patient(self, patient_id, name, severity_level, admission_date, bill_amount):
75          patient = Patient(patient_id, name, severity_level, admission_date, bill_amount)
76          self.patients.append(patient)
77
78      def search_patient_by_id(self, patient_id):
79          for patient in self.patients:
80              if patient.patient_id == patient_id:
81                  return patient
82          return None
83
84      def search_patient_by_name(self, name):
85          for patient in self.patients:
86              if patient.name.lower() == name.lower():
87                  return patient
```

Ass-12.5.py > Patient

```
70  class HospitalRecords:
84      def search_patient_by_name(self, name):
88          return None
89
90      def sort_patients_by_severity_level(self):
91          self.patients.sort(key=lambda x: x.severity_level)
92
93      def sort_patients_by_bill_amount(self):
94          self.patients.sort(key=lambda x: x.bill_amount)
95  # Example usage
96  hospital = HospitalRecords()
97  hospital.add_patient("PT001", "John Doe", 5, "2024-01-01", 2000.00)
98  hospital.add_patient("PT002", "Jane Smith", 3, "2024-01-02", 1500.00)
99  hospital.add_patient("PT003", "Alice Johnson", 4, "2024-01-03", 3000.00)
100  print("Search for patient ID PT001:", hospital.search_patient_by_id("PT001"))
101  print("Search for patient name 'Jane Smith':", hospital.search_patient_by_name("Jane Smith"))
102  hospital.sort_patients_by_severity_level()
103  print("Patients sorted by severity level:", hospital.patients)
104  hospital.sort_patients_by_bill_amount()
105  print("Patients sorted by bill amount:", hospital.patients)
106  #justification for selected algorithm:
107  # Similar to the retail store's inventory system, we can use a linear search for finding patient
108  # records by ID or name due to the manageable dataset size and the need for simplicity. The time complexity
109  # of linear search is O(n) in the worst case. For sorting patients by severity level or bill amount, we can again
110  # use the built-in sort method which implements Timsort. Timsort is efficient for real-world data and has a time
111  # complexity of O(n log n) in the worst case and O(n) in the best case when the data is already sorted. This
112  # makes it suitable for our hospital records system where we may often have partially sorted data and need
113  # efficient sorting for prioritization and billing.
114
```

OUTPUT:


```
▼ TERMINAL powershell + v [ ] [ ] ...
at 0x000001FE01F48B90>, <__main__.Product object at 0x000001FE01F48A50>]
PS C:\Users\reddy\OneDrive\Desktop\AI> & C:\Users\reddy\AppData\Local\Microsoft\WindowsApps\python3.13.exe c:
/Users/reddy/OneDrive/Desktop/AI/Ass-12.5.py
Search for product ID P001: <__main__.Product object at 0x000001C347836BA0>
Search for product name 'Smartphone': <__main__.Product object at 0x000001C347AC8A50>
Products sorted by price: [<__main__.Product object at 0x000001C347AC8B90>, <__main__.Product object at 0x000
001C347AC8A50>, <__main__.Product object at 0x000001C347836BA0>]
Products sorted by stock quantity: [<__main__.Product object at 0x000001C347836BA0>, <__main__.Product object
at 0x000001C347AC8B90>, <__main__.Product object at 0x000001C347AC8A50>]
Search for patient ID PT001: <__main__.Patient object at 0x000001C347836E40>
Search for patient name 'Jane Smith': <__main__.Patient object at 0x000001C347AC8CD0>
Patients sorted by severity level: [<__main__.Patient object at 0x000001C347AC8CD0>, <__main__.Patient object
at 0x000001C347AC8E10>, <__main__.Patient object at 0x000001C347836E40>]
Patients sorted by bill amount: [<__main__.Patient object at 0x000001C347AC8CD0>, <__main__.Patient object at
0x000001C347836E40>, <__main__.Patient object at 0x000001C347AC8E10>]
PS C:\Users\reddy\OneDrive\Desktop\AI> █
```

Task Description #5: University Examination Result Processing System

A university processes examination results for thousands of students containing roll number, name, subject, and marks. The system must:

1. Search student results using roll number.
2. Sort students based on marks to generate rank lists.

Student Task

- Identify efficient searching and sorting algorithms using AI assistance.
- Justify the choice of algorithms.
- Implement the algorithms in Python.

```
... Welcome task5.py
C:\Users\varshitha\OneDrive\Desktop\AI Assistant coding> task5.py > ...
1 #Generate a code A university processes examination results for thousands of students containing roll number, name, subject, and marks
2 #Search student results using roll number.Sort students based on marks to generate rank lists.
3 # University Examination Result Processing System
4
5 class StudentResult:
6     def __init__(self, roll_number, name, subject, marks):
7         self.roll_number = roll_number
8         self.name = name
9         self.subject = subject
10        self.marks = marks
11
12    def __repr__(self):
13        return f"StudentResult({self.roll_number}, {self.name}, {self.subject}, {self.marks})"
14
15
16 class ExaminationSystem:
17     """
18     University examination result processing system.
19     Uses binary search for efficient searching and merge sort for sorting.
20     """
21
22    def __init__(self):
23        self.results = []
24
25    def add_result(self, roll_number, name, subject, marks):
26        """Add a student result."""
27        self.results.append(StudentResult(roll_number, name, subject, marks))
28
29    def binary_search_by_roll(self, roll_number):
30        """
31        Binary search for student by roll number.
32        Time Complexity: O(log n)
33        Justification: Efficient for large datasets; requires sorted data.
34        """
35        sorted_results = sorted(self.results, key=lambda x: x.roll_number)
36        left, right = 0, len(sorted_results) - 1
```

```

38         while left <= right:
39             mid = (left + right) // 2
40             if sorted_results[mid].roll_number == roll_number:
41                 return sorted_results[mid]
42             elif sorted_results[mid].roll_number < roll_number:
43                 left = mid + 1
44             else:
45                 right = mid - 1
46
47         return None
48
49     def merge_sort_by_marks(self):
50         """
51         Merge sort students by marks in descending order.
52         Time Complexity: O(n log n)
53         Justification: Stable sort, consistent performance, efficient for large datasets.
54         """
55         def merge(arr, left, mid, right):
56             left_arr = arr[left:mid+1]
57             right_arr = arr[mid+1:right+1]
58             i = j = 0
59             k = left
60
61             while i < len(left_arr) and j < len(right_arr):
62                 if left_arr[i].marks >= right_arr[j].marks:
63                     arr[k] = left_arr[i]
64                     i += 1
65                 else:
66                     arr[k] = right_arr[j]
67                     j += 1
68                 k += 1
69
70             while i < len(left_arr):
71                 arr[k] = left_arr[i]

```

```

49     def merge_sort_by_marks(self):
55         def merge(arr, left, mid, right):
60
70             while i < len(left_arr):
71                 arr[k] = left_arr[i]
72                 i += 1
73                 k += 1
74
75             while j < len(right_arr):
76                 arr[k] = right_arr[j]
77                 j += 1
78                 k += 1
79
80         def merge_sort_helper(arr, left, right):
81             if left < right:
82                 mid = (left + right) // 2
83                 merge_sort_helper(arr, left, mid)
84                 merge_sort_helper(arr, mid + 1, right)
85                 merge(arr, left, mid, right)
86
87         if self.results:
88             merge_sort_helper(self.results, 0, len(self.results) - 1)
89
90         return self.results
91
92     def generate_rank_list(self):
93         """Generate rank list sorted by marks."""
94         sorted_results = self.merge_sort_by_marks()
95         return [(i+1, result) for i, result in enumerate(sorted_results)]
96
97     def display_rank_list(self):
98         """Display rank list with ranks."""
99         rank_list = self.generate_rank_list()
100         print("\n{'RANK':<6}{'ROLL NO':<12}{'NAME':<20}{'SUBJECT':<15}{'MARKS':<8}")
101         print("-" * 65)

```



```

97     def display_rank_list(self):
98         for rank, result in rank_list:
99             print(f"rank:<6}{result.roll_number:<12}{result.name:<20}{result.subject:<15}{result.marks:<8}")
100
101 # Example usage
102 if __name__ == "__main__":
103     system = ExaminationSystem()
104
105     # Add student results
106     system.add_result(101, "Alice Johnson", "Mathematics", 95)
107     system.add_result(102, "Bob Smith", "Mathematics", 87)
108     system.add_result(103, "Charlie Brown", "Mathematics", 92)
109     system.add_result(104, "Diana Prince", "Mathematics", 88)
110     system.add_result(105, "Eve Davis", "Mathematics", 90)
111
112     # Search by roll number
113     print("Searching for Roll Number 103:")
114     result = system.binary_search_by_roll(103)
115     print(result if result else "Not found")
116
117     # Generate and display rank list
118     print("\nGenerated Rank List:")
119     system.display_rank_list()

```

OUTPUT:

```

PS C:\Users\varshitha> & C:/Users/varshitha/anaconda3/python.exe "c:/Users/varshitha/OneDrive/Desktop/AI Assistant coding/task5.py"
Searching for Roll Number 103:
StudentResult(103, Charlie Brown, Mathematics, 92)

Generated Rank List:

{'RANK':<6>{'ROLL NO':<12>{'NAME':<20>{'SUBJECT':<15>{'MARKS':<8>}}}}
-----
1    101    Alice Johnson    Mathematics    95
2    103    Charlie Brown    Mathematics    92

{'RANK':<6>{'ROLL NO':<12>{'NAME':<20>{'SUBJECT':<15>{'MARKS':<8>}}}}
-----
1    101    Alice Johnson    Mathematics    95
2    103    Charlie Brown    Mathematics    92
3    105    Eve Davis        Mathematics    90
4    104    Diana Prince     Mathematics    88
5    102    Bob Smith        Mathematics    87
PS C:\Users\varshitha>

```

Task Description #6: Online Food Delivery Platform

An online food delivery application stores thousands of orders with order ID, restaurant name, delivery time, price, and order status. The platform needs to:

1. Quickly find an order using order ID.
2. Sort orders based on delivery time or price.

Student Task

- Use AI to suggest optimized algorithms.
- Justify the algorithm selection.
- Implement searching and sorting modules in Python.

```
... Welcome task6.py
C: > Users > varshitha > OneDrive > Desktop > AI Assistant coding > task6.py > ...
1 #An online food delivery application stores thousands of orders with order ID, restaurant name, delivery time, price, and order status.
2 #platform needs to:1. Quickly find an order using order ID.2. Sort orders based on delivery time or price.
3
4 from dataclasses import dataclass
5 from typing import List
6 from datetime import datetime
7
8 @dataclass
9 class Order:
10     order_id: int
11     restaurant_name: str
12     delivery_time: int # in minutes
13     price: float
14     status: str
15
16 class FoodDeliveryPlatform:
17     def __init__(self):
18         self.orders: List[Order] = []
19         self.order_index: dict = {} # Hash map for O(1) search
20
21     def add_order(self, order: Order) -> None:
22         """Add order and maintain index for fast lookup."""
23         self.orders.append(order)
24         self.order_index[order.order_id] = order
25
26     def search_by_order_id(self, order_id: int) -> Order:
27         """
28         Search order by ID using hash map.
29         Time Complexity: O(1) - Constant time lookup
30         Justification: Hash map provides instant access without scanning.
31         """
32         return self.order_index.get(order_id, None)
33
34     def sort_by_delivery_time(self) -> List[Order]:
35         """
36         Sort orders by delivery time using Timsort.
37         Time Complexity: O(n log n) - Python's built-in optimal
```

```
38         Justification: Timsort handles partially sorted data efficiently.
39         """
40         return sorted(self.orders, key=lambda x: x.delivery_time)
41
42     def sort_by_price(self) -> List[Order]:
43         """
44         Sort orders by price using Timsort.
45         Time Complexity: O(n log n)
46         Justification: Best for general-purpose sorting of comparable elements.
47         """
48         return sorted(self.orders, key=lambda x: x.price)
49
50     def filter_and_sort(self, status: str, sort_by: str) -> List[Order]:
51         """Filter by status and sort by delivery_time or price."""
52         filtered = [o for o in self.orders if o.status == status]
53
54         if sort_by == "delivery_time":
55             return sorted(filtered, key=lambda x: x.delivery_time)
56         elif sort_by == "price":
57             return sorted(filtered, key=lambda x: x.price)
58         return filtered
59
60
61 # Example Usage
62 if __name__ == "__main__":
63     platform = FoodDeliveryPlatform()
64
65     # Add sample orders
66     platform.add_order(Order(101, "Pizza Hut", 30, 12.99, "delivered"))
67     platform.add_order(Order(102, "Burger King", 45, 8.50, "pending"))
68     platform.add_order(Order(103, "Subway", 20, 9.75, "delivered"))
69     platform.add_order(Order(104, "McDonald's", 35, 6.99, "in_progress"))
70
```

```
C: > Users > varshitha > OneDrive > Desktop > AI Assistant coding > task6.py > ...

71     # Search by order ID
72     order = platform.search_by_order_id(102)
73     print(f"Found Order: {order}\n")
74
75     # Sort by delivery time
76     print("Sorted by Delivery Time:")
77     for order in platform.sort_by_delivery_time():
78         print(f" {order.order_id}: {order.delivery_time} min")
79
80     # Sort by price
81     print("\nSorted by Price:")
82     for order in platform.sort_by_price():
83         print(f" {order.order_id}: ${order.price}")
```

OUTPUT:

```
PS C:\Users\varshitha> & C:/Users/varshitha/anaconda3/python.exe "c:/Users/varshitha/OneDrive/Desktop/AI Assistant coding/task6.py"
Found Order: Order(order_id=102, restaurant_name='Burger King', delivery_time=45, price=8.5, status='pending')

Sorted by Delivery Time:
103: 20 min
101: 30 min
104: 35 min
102: 45 min

Sorted by Price:
104: $6.99
102: $8.5
103: $9.75

Sorted by Delivery Time:
103: 20 min
101: 30 min
104: 35 min
102: 45 min

Sorted by Price:
104: $6.99
102: $8.5
103: $9.75

Sorted by Delivery Time:
103: 20 min
101: 30 min
104: 35 min
102: 45 min

Sorted by Price:
104: $6.99
102: $8.5
103: $9.75
101: 30 min
104: 35 min
102: 45 min

Sorted by Price:
104: $6.99
102: $8.5
```

```
Sorted by Price:
104: $6.99
102: $8.5
103: $9.75
102: 45 min

Sorted by Price:
104: $6.99
102: $8.5
103: $9.75
104: $6.99
102: $8.5
103: $9.75
102: $8.5
103: $9.75
101: $12.99
PS C:\Users\varshitha>
```